

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи N10
з дисципліни “Об'єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Ліviv 2026

Тема: Модульне тестування об'єктно-орієнтованих програм у Python.

Мета: Навчитися створювати тести для класів і функцій за допомогою бібліотеки `unittest`, використовуючи параметризовані тести та мок-об'єкти.

Завдання:

Реалізувати модульне тестування.

Основні вимоги:

- написати тести для функцій і класів;
- використовувати assert-методи;
- реалізувати setUp/tearDown;
- застосувати параметризовані тести;
- використати mock-об'єкти.

Код:

math_tool:

```
class MathTool:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Ділення на нуль")
        return a / b
```

test_math_tool:

```

import unittest
from math_tool import MathTool

class TestMathTool(unittest.TestCase):
    new *

    def setUp(self):
        new *
        self.math = MathTool()

    def test_add(self):
        new *
        self.assertEqual(self.math.add(a: 2, b: 3), second: 5)

    def test_subtract(self):
        new *
        self.assertEqual(self.math.subtract(a: 5, b: 3), second: 2)

    def test_multiply(self):
        new *
        self.assertEqual(self.math.multiply(a: 4, b: 3), second: 12)

    def test_divide(self):
        new *
        self.assertEqual(self.math.divide(a: 10, b: 2), second: 5)

    def test_divide_by_zero(self):
        new *
        with self.assertRaises(ValueError):
            self.math.divide(a: 10, b: 0)

if __name__ == "__main__":
    unittest.main()

```

library.py:

```

class LibraryItem:
    6 usages
    new *

    def __init__(self, title, author, year):
        new *
        self.title = title
        self.author = author
        self.year = year

    def details(self):
        4 usages
        new *
        return f"{self.title} - {self.author} ({self.year})"

```

test_library.py:

```

import unittest
from library import LibraryItem

class TestLibraryItem(unittest.TestCase):
    new *

    def test_details(self):
        new *
        item = LibraryItem(title: "1984", author: "George Orwell", year: 1949)
        self.assertEqual(item.details(), second: "1984 - George Orwell (1949)")

        item2 = LibraryItem(title: "Python", author: "Guido van Rossum", year: 1991)
        self.assertEqual(item2.details(), second: "Python - Guido van Rossum (1991)")

if __name__ == "__main__":
    unittest.main()

```

notification.py:

```

class NotificationService: 2 usages new *
    def send(self, user, message): 3 usages (3 dynamic) new *
        pass

class UserManager: 2 usages new *
    def __init__(self, service): new *
        self.service = service

    def notify_user(self, user, message): 1 usage new *
        self.service.send(user, message)]

```

test_notification.py:

```

import unittest
from unittest.mock import Mock
from notification import NotificationService, UserManager

class TestUserManager(unittest.TestCase): new *

    def test_notify_user(self): new *
        mock_service = Mock(spec=NotificationService)

        manager = UserManager(mock_service)
        manager.notify_user(user="Ivan", message="Hello!")

        mock_service.send.assert_called_once_with("Ivan", "Hello!")

if __name__ == "__main__":
    unittest.main()]

```

utils.py:

```

def check_even(number): 2 usages new *
    return number % 2 == 0

```

test_utils.py:

```

import unittest
from parameterized import parameterized
from utils import check_even

class TestCheckEven(unittest.TestCase): new *

    @parameterized.expand([ new *
        ("positive_even", 2, True),
        ("positive_odd", 3, False),
        ("zero", 0, True),
        ("negative_even", -4, True),
        ("negative_odd", -5, False),
    ])
    def test_check_even(self, name, number, expected):
        self.assertEqual(check_even(number), expected)

if __name__ == "__main__":
    unittest.main()]

```

Результат:

```
✓ 1 test passed 1 test total, 1ms
C:\Users\User\AppData\Local\Programs
Testing started at 16:54 ...
Launching unittests with arguments p

Ran 1 test in 0.002s

OK

Process finished with exit code 0
```

Висновок: Модульне тестування є критично важливим для забезпечення якості коду. Використання Mock-об'єктів дозволяє ізольовано тестувати компоненти, а методи `setUp` та `tearDown` допомагають автоматизувати підготовку та очищення середовища для кожного тесту.